Data Integration Guide — Receiving Simulator Data on Your Own Device

This guide is for partners who want to collect the data our simulator sends (lap times, driver name, distance traveled, lead-gen survey answers, contact info) in their own application, alongside or instead of our leaderboard display.

How data leaves the simulator

The simulator's **Lap Time Sender** application pushes data over plain **HTTP POST requests with JSON bodies** on your local network. (UDP is only used for optional auto-discovery — see the end of this document.)

The Sender supports **multiple receivers at once**: every configured receiver gets an identical copy of every POST. To receive the data, your app just needs to:

1. Run a small HTTP server on the same network (our default port is **5000**, but any port works).
2. Have its URL added in the Sender's **Server URL** field, comma-separated with the leaderboard, e.g.:

`` `http://192.168.1.50:5000, http://192.168.1.99:5000` ``

3. Respond `200 OK` (an empty body is fine) within 5 seconds.

That's the whole integration. Everything below describes what you'll receive.

---

1. Lap / run results — `POST /`

Sent the moment a lap completes (or, in sector-challenge mode, the moment a run ends). This is the main event your app should store.

```
{
  "simulator_id": 1,
  "driver_name": "Jane Driver",
  "lap_time": 512.417,
  "formatted_time": "8:32.417",
  "session_id": "0b1f8a52-6f0e-4a7e-9c93-1f2ab36c9d10",
  "email": "jane@example.com",
  "phone": "+15551234567",
  "distance_pct": 100.0,
  "survey_answers": {
    "How much money does your company manage?": "50-100M",
    "How did you hear about us?": "Event"
  }
}
```

Field	Type	Always present?	Meaning
`simulator_id`	int	yes	Which simulator sent the result (1, 2, 3, …)
`driver_name`	string	yes	Name entered at registration

Field	Type	Always present?	Meaning
`lap_time`	float (seconds)	yes	Lap time; for a *failed* sector-challenge run this is the elapsed time when the run ended
`formatted_time`	string	usually	Same time as `M:SS.mmm` for display
`session_id`	string (UUID)	yes	Stable for one driver's stint; regenerated per driver. **Use `(session_id, lap_time)` to deduplicate** (see Delivery semantics)
`email`	string	when Lead Gen is on	Driver's email from registration
`phone`	string	when Lead Gen is on	Driver's phone in E.164 format (`+1…`)
`distance_pct`	float 0–100	sector-challenge mode only	Distance traveled around the track. `100` = finished the lap; less = run failed at that point. Rank by distance descending, then `lap_time` ascending
`survey_answers`	object	when survey questions are configured	Lead-gen questionnaire: each key is the question text, each value the chosen option

Notes:

- Treat unknown/extra fields as ignorable — new optional fields may be added.
- `survey_answers` reflects the questions configured at the time of registration; question text is the key, so your storage should handle arbitrary keys.

2. Session start — `POST /api/session/started` (optional to handle)

Sent when a driver registers and their stint begins. Useful if you want the registration record even for drivers who never complete a lap.

```
{
  "simulator_ip": "192.168.1.20",
  "driver_name": "Jane Driver",
  "queue_id": "",
  "session_id": "0b1f8a52-6f0e-4a7e-9c93-1f2ab36c9d10",
  "email": "jane@example.com",
  "phone": "+15551234567",
  "survey_answers": { "How did you hear about us?": "Event" }
}
```

3. Session end — `POST /api/session/ended` (optional to handle)

```
{ "simulator_ip": "192.168.1.20", "duration_seconds": 285 }
```

If your app doesn't care about sessions, just return `200` for these paths (or return `404` — the Sender ignores failures on session notifications).

Delivery semantics (important)

- **Timeout**: each POST times out after 5 seconds. Keep your handler fast; store the JSON and return immediately.
- **Retries / duplicates**: if *all* receivers fail, the lap is queued and retried every 30 seconds. Your endpoint may therefore receive the same lap more than once (e.g. it succeeded but your `200` was lost). Deduplicate on `(session_id, lap_time)`.
- **Ordering**: laps arrive in order per simulator under normal conditions, but a retried lap can arrive after a newer one. Sort by your own receive time or the lap data itself, not arrival order.
- **Network**: all traffic is plain HTTP on the local network — your device must be on the same LAN/subnet as the simulators (or routable from them). Make sure your firewall allows inbound connections on your chosen port.

Optional: auto-discovery (skip if you enter the URL manually)

Our components find each other with a UDP broadcast handshake. If your app implements it, the Sender will add your app automatically without anyone typing a URL:

1. Listen on **UDP port 5001**.
2. When you receive the datagram `LEADERBOARD_DISCOVER` (ASCII), reply to the sender's address with: `LEADERBOARD_SERVER:<your_ip>:<your_http_port>` — e.g. `LEADERBOARD_SERVER:192.168.1.99:5000`.
3. The Sender then starts POSTing to `http://<your_ip>:<your_http_port>`.

Minimal working receiver (Python)

```
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
import json, csv, os

CSV_FILE = "collected_data.csv"

class Handler(BaseHTTPRequestHandler):
    def do_POST(self):
        body = self.rfile.read(int(self.headers.get("Content-Length", 0)))
        data = json.loads(body or b"{}")
        if self.path == "/":  # lap/run result
            flat = {k: v for k, v in data.items() if k != "survey_answers"}
            flat.update(data.get("survey_answers", {}))
            write_header = not os.path.exists(CSV_FILE)
            with open(CSV_FILE, "a", newline="", encoding="utf-8") as f:
                w = csv.DictWriter(f, fieldnames=list(flat.keys()), extrasaction="ignore"
                if write_header:
                    w.writeheader()
                w.writerow(flat)
        self.send_response(200)
        self.end_headers()

ThreadingHTTPServer(("", 5000), Handler).serve_forever()
```

(A production app should use a fixed column list and deduplicate on `(session_id, lap_time)` — this is just the smallest thing that works.)

Data reference summary

Data	Where it arrives
Lap time / run elapsed time	`POST /` → `lap_time`, `formatted_time`
Driver name	`POST /` and `/api/session/started` → `driver_name`
Distance traveled (sector challenge)	`POST /` → `distance_pct`
Email / phone	`POST /` and `/api/session/started` (Lead Gen must be enabled)
Survey question answers	`POST /` and `/api/session/started` → `survey_answers`
Session start/end times	`/api/session/started`, `/api/session/ended`